Clustering Techniques

Applied Machine Learning — Session 3, Chapter 2

What Is Clustering?

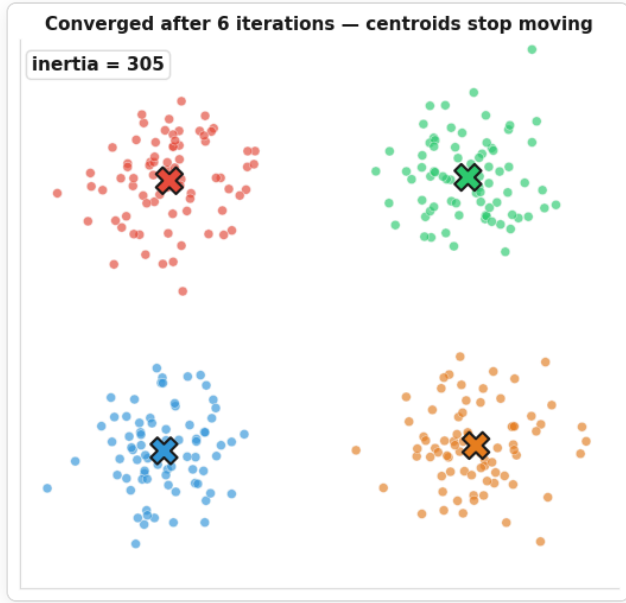
Partition data into groups such that:

- samples **within** a cluster are similar
- samples in **different** clusters are dissimilar

No labels — we discover the groups. The output is a new column `labels`.

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)      # array([0, 2, 1, 0, ...]) — no y anywhere
```

K-Means: The Algorithm



1. pick k random points as centroids → **2.** ASSIGN each point to its nearest centroid → **3.** UPDATE each centroid to the mean of its points → repeat 2–3 until nothing moves.

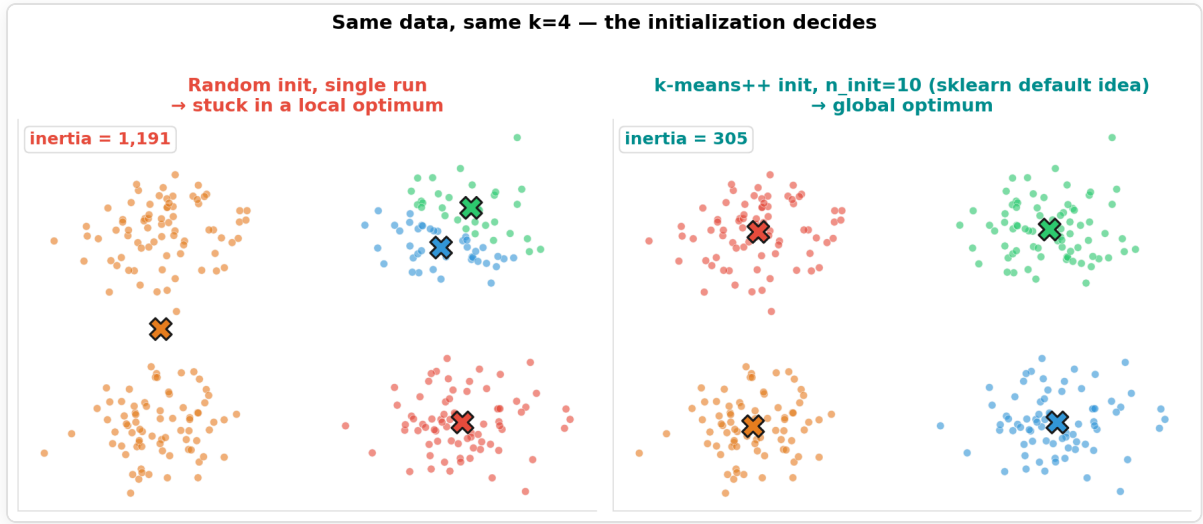
K-Means: The Parameters

```
from sklearn.cluster import KMeans

kmeans = KMeans(
    n_clusters=3,        # k — the most important choice (next slides)
    init='k-means++',    # smart spread-out initialization (default)
    n_init=10,           # run 10 times, keep the best (lowest inertia)
    random_state=42,     # reproducible
)
kmeans.fit(X)

kmeans.labels_           # cluster id per sample
kmeans.cluster_centers_  # centroid coordinates
kmeans.inertia_          # within-cluster sum of squares (lower = tighter)
```

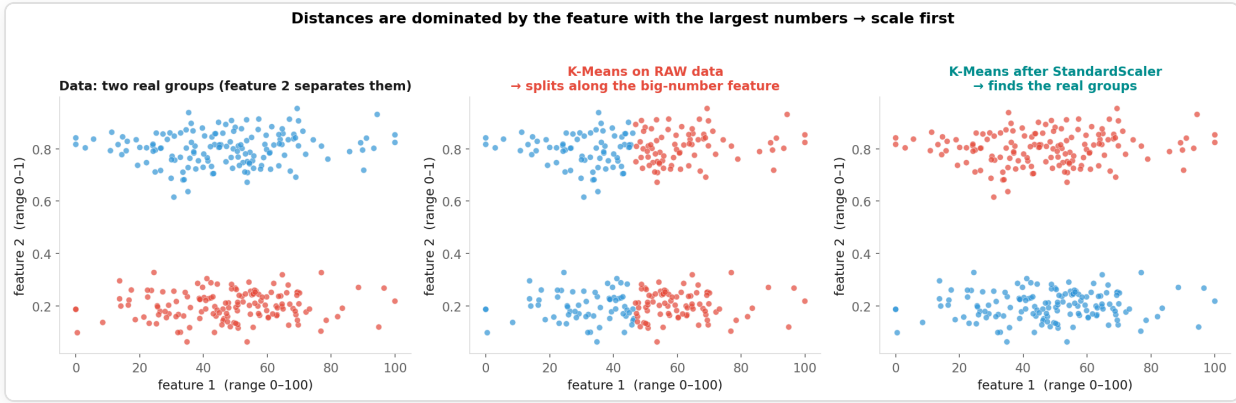
Why `init` and `n_init` Matter



K-Means only finds a **local** optimum. Different starts → different results.

`k-means++` spreads the initial centroids out; `n_init=10` keeps the best of 10 runs.

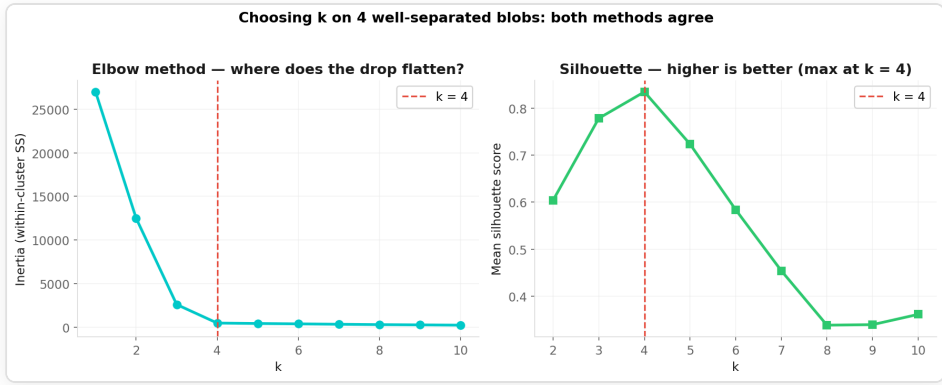
Scale First! (Distance-Based = Scale-Sensitive)




```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pipe = make_pipeline(StandardScaler(), KMeans(n_clusters=2, n_init=10, random_state=42))
labels = pipe.fit_predict(X)          # scaler fitted inside — same idiom as Session 2
```

Applies to **K-Means, DBSCAN, hierarchical** — anything that computes distances (KNN, Ch03!).

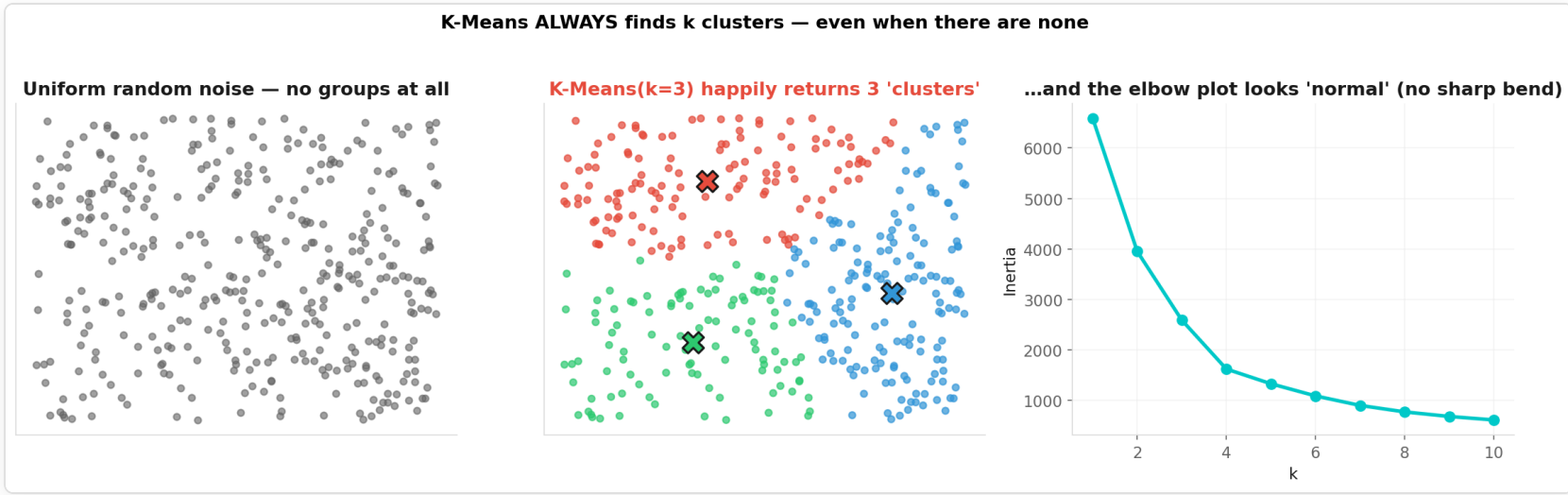
How to Choose k? — Elbow & Silhouette




```python
from sklearn.metrics import silhouette_score
for k in range(2, 9):
    labels = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X_scaled)
    print(k, silhouette_score(X_scaled, labels))   # a = dist to own cluster, b = to nearest
```

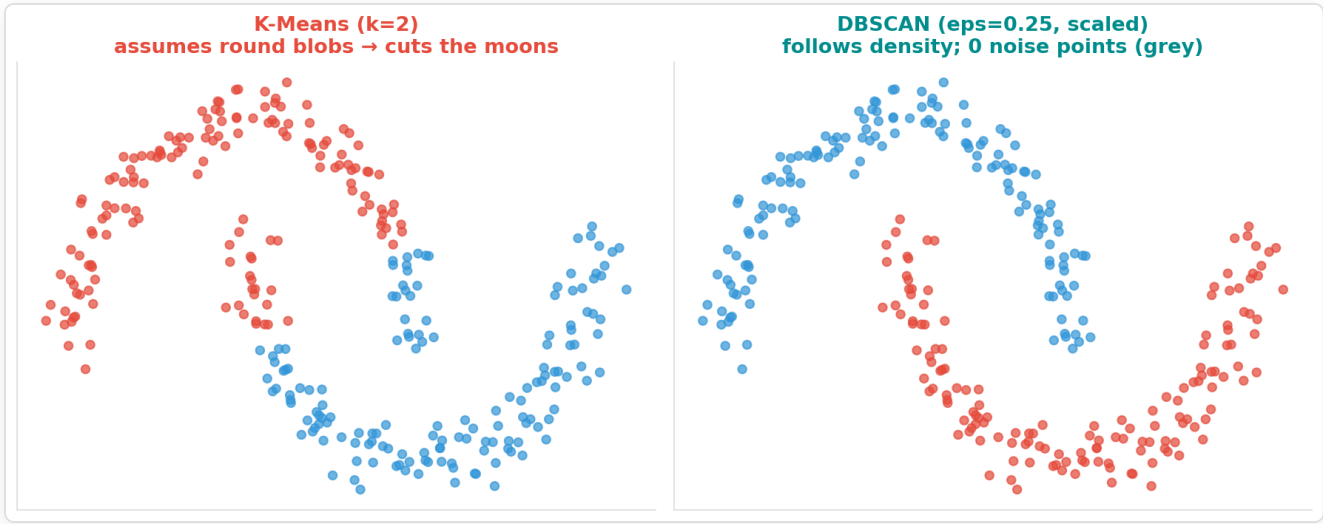
Silhouette = (b − a) / max(a, b) per sample, averaged: +1 well separated, 0 on a boundary, −1 wrong cluster.

⚠️ Clustering Always "Works" — Even When It Shouldn't



Before trusting clusters: **look at the data (PCA 2-D, Ch09)**, check silhouette (≈0.38 here vs. 0.83 for the real blobs), and ask whether the groups mean anything.

DBSCAN: Density-Based Clustering



`labels = DBSCAN(eps=0.25, min_samples=5).fit_predict(X_scaled)` — **core point** = ≥ `min_samples` neighbours within `eps`; chains of core points form a cluster; the rest is **noise (−1)** → built-in outlier detection.

Which Algorithm?

Algorithm	Cluster shape	k needed	Outliers	Notes
K-Means	round, similar size	yes	pulled by them	fast, default first try
DBSCAN	any (density)	no	flagged as −1	eps tuning; struggles with mixed densities
Hierarchical *(bonus)*	any	no (cut tree)	no	dendrogram, $O(n^2)$ — small data
GMM *(mention)*	ellipses, soft	yes	no	probabilities instead of hard labels

Rule of thumb: K-Means first → check silhouette + plot → DBSCAN if shapes are odd or you want outliers.

Quick Check

Q1. Your elbow plot suggests k=3, the silhouette score is highest at k=2. What do you do?

Q2. You cluster customers by `age` (18–80) and `income` (20 000–200 000) without scaling. What will K-Means effectively cluster on?

Quick Check

Q1. Your elbow plot suggests k=3, the silhouette score is highest at k=2. What do you do?

→ Neither is an oracle. Look at both clusterings (plot!), ask which is more *useful* for your question; report that the 3-cluster solution splits one of the two big groups. Both are legitimate answers.

Q2. You cluster customers by `age` (18–80) and `income` (20 000–200 000) without scaling. What will K-Means effectively cluster on?

Quick Check

Q1. Your elbow plot suggests k=3, the silhouette score is highest at k=2. What do you do?

→ Neither is an oracle. Look at both clusterings (plot!), ask which is more *useful* for your question; report that the 3-cluster solution splits one of the two big groups. Both are legitimate answers.

Q2. You cluster customers by `age` (18–80) and `income` (20 000–200 000) without scaling. What will K-Means effectively cluster on?

→ Almost only on income — its differences are ~1000× larger, so it dominates the distance. Scale first (Pipeline with StandardScaler).

Live Demo

→ Open `02-examples/ch08_clustering_examples.ipynb` (~8 min)

1. Elbow + silhouette on blobs — and on **pure noise**
2. K-Means vs DBSCAN on moons
3. Customer segmentation: scale → K-Means → **profile** the segments → name them from the numbers

Now: Exercises!

→ Open `03-exercises/ch08_clustering_exercises.ipynb`

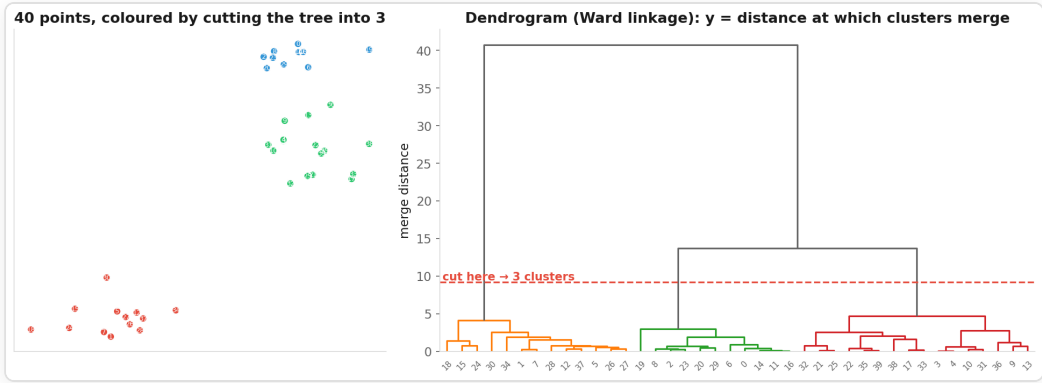
Task: Cluster the Iris flowers **without** the species labels:
elbow → silhouette → decide k (they disagree!) → fit in a Pipeline → plot → *sanity-check* against species.

~10 minutes · Bonus: DBSCAN, hierarchical clustering

Key Takeaways

- K-Means: assign → update → repeat; finds a **local** optimum → `k-means++` , `n_init`
- **Scale first** — every distance-based method
- Choosing k: elbow + silhouette + **plot** + domain sense — no oracle
- K-Means always returns k clusters — check whether the structure is real
- DBSCAN: density, no k, arbitrary shapes, outliers = −1
- Clusters ≠ classes: label agreement is a sanity check, not the objective

Bonus / Appendix: Hierarchical Clustering




```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
Z = linkage(X_scaled, method='ward')          # bottom-up merges: closest clusters first
dendrogram(Z)                                 # y-axis = distance at which two clusters merged
labels = fcluster(Z, t=3, criterion='maxclust') - 1   # cut the tree into 3 clusters
```

Long vertical branches = natural gaps → good places to cut. Ward linkage minimises within-cluster variance (K-Means' cousin). O(n^2) memory → small datasets only.

Next: Chapter 9

Dimensionality Reduction

"Sometimes 2 dimensions reveal more than 100."